

ADVANCED DECISION STRUCTURES

Nested Logic & Switch Case Systems

CodeHive Academy

Master Level Logic Series

1. Nested If Statements

Sometimes, one condition isn't enough. You need to check a condition inside another condition. This is called **Nesting**. Use this when a second check only matters if the first one is true.

```
int isLoggedIn = 1; // 1 for true int isPremium = 0; // 0 for false if
(isLoggedIn == 1) { if (isPremium == 1) { cout << "Welcome to the Premium
Dashboard!"; } else { cout << "Welcome! Upgrade to Premium for more
features."; } } else { cout << "Please log in to continue."; }
```

Usage: Ideal for hierarchical data checks, like verifying an account exists before checking its permissions.

2. Logical Operators (The "AND" Shortcut)

Instead of deep nesting, you can use Logical Operators to keep your CodeHive projects clean and readable. This is often called "flattening" your logic.

- **&& (AND):** Both sides must be true.
- **|| (OR):** At least one side must be true.
- **! (NOT):** Flips the result (true becomes false).

```
int age = 22; bool hasLicense = true; if (age ≥ 18 && hasLicense) { cout  
<< "You are allowed to drive!"; }
```

3. The Switch Statement

If you have a variable that could be one of 10 different values (like a menu choice), using 10 `else if` statements looks messy. The **switch** statement is a cleaner, more optimized alternative.

How it works: The program compares the variable to several "cases". If a match is found, the code under that case runs.

The 'break' keyword

This is vital. It tells C++ to "break out" of the switch block once a match is successfully handled. Without it, the program will continue into the next cases (even if they don't match!).

4. Real-Life Example (Navigation Menu)

On your coding platform sidebar, a switch can handle menu selections efficiently:

```
int menuChoice = 2; switch (menuChoice) { case 1: cout << "Opening  
Profile ... "; break; case 2: cout << "Opening Settings ... "; break; case 3:  
cout << "Logging out ... "; break; default: cout << "Invalid Selection."; }
```

5. The "Default" Safety Net

The `default` keyword is used to run code if **none** of the cases match your variable. It is your ultimate fallback plan.

Concept: Think of it as the final `else` in an if-else chain.

Best Practices:

- Always include a default case to handle unexpected user input or malicious data.
- Use descriptive error messages in default cases to help CodeHive users debug their actions.

6. Practice Challenge: The Switch Calculator

Open your IDE and build a functional calculator using what you've learned. This trains your ability to handle char inputs.

Task Requirements:

1. Input two numbers (float).
2. Input a character operator (+, -, *, /).
3. Use a switch statement on the operator to perform the math.
4. Handle division by zero in the '/' case!

7. Practice Challenge: Grade Classifier

Create a program that helps the CodeHive Academy grading system categorize results.

Input: A score from 0-100.

Conditions:

- If score > 90: Print "Excellent".
- If score 70-90: Print "Good".
- If score < 70: Print "Keep Practicing".

Hint: Since scores are ranges, use concentrated `if...else if` logic here instead of switch.

8. Common Mistakes & Summary

Before moving to loops, avoid these "Gotchas":

- **Forgetting break:** Leads to "Fall-through" errors.
- **Using expressions:** You cannot use `case > 50:` in a standard switch. Switch checks exact values.
- **Case Sensitivity:** 'A' is not the same as 'a' in C++.

Code. Learn. Repeat.

